Основе дистрибуираног програмирања

Практикум

0.1 Како користити практикум

Практикум је самосталан материјал за разумевање дистрибуираних система. Свака вежба објашњава проблем који расподела система уводи, разлог због ког је изабрани принцип важан, типичне грешке и начин на који се одлука проверава у развоју софтвера. Циљ је да студент након вежбе може самостално да обнови принцип и примени га на додељеној пројектној целини, а не да запамти један конкретан пример.

Од методе до пројекта

Практикум не даје готово дистрибуирано решење за пројекат. Сваки тим добија додељену пројектну целину (нпр. телеметрију, команде, координацију или репликацију) и принцип са вежбе примењује на сопствени домен, уз образложење одлуке на пројектној контролној тачки.

Табела 1. Фаза и Препоручени начин рада

Фаза	Препоручени начин рада
Пре вежбе	Прочитати тему и означити претпоставке које мрежа, време или више инстанци могу да покваре.
Током вежбе	Пратити разлог одлуке и њен компромис, а не само коначни облик имплементације.
После вежбе	Повезати принцип са конкретним дистрибуираним сценаријем и објаснити доказ кроз тест или контролисани сценарио отказа.
Пред одбрану	Објаснити границу одговорности, неизвестан исход и разлог због ког систем на њега реагује баш тако.

Извршиви примери

Сваку вежбу прати мали, самосталан Ц шарп и дотнет пример из директоријума `examples/odp-dotnet`. Пример није део пројекта: он на једној одлуци показује принцип, сценарио отказа и тест који тај сценарио задаје. Одељак „Извршиви пример“ на крају сваке вежбе наводи правило у коду, тест и испис демонстрације.

Симулатори уместо реалне опреме

Практикум не захтева реалну орбиталну механику, радио опрему ни праву сателитску комуникацију. Удаљени уређаји, мрежа и отказ ситуације репродукују се контролисаним, поновљивим симулаторима.

Језик и алати

Примери су претежно у Ц шарп и дотнет окружењу. Друга технологија може бити одобрена када тим обезбеди интероперабилност и еквивалентан ниво тестирања и отказ покривености.

0.2 Ток градива

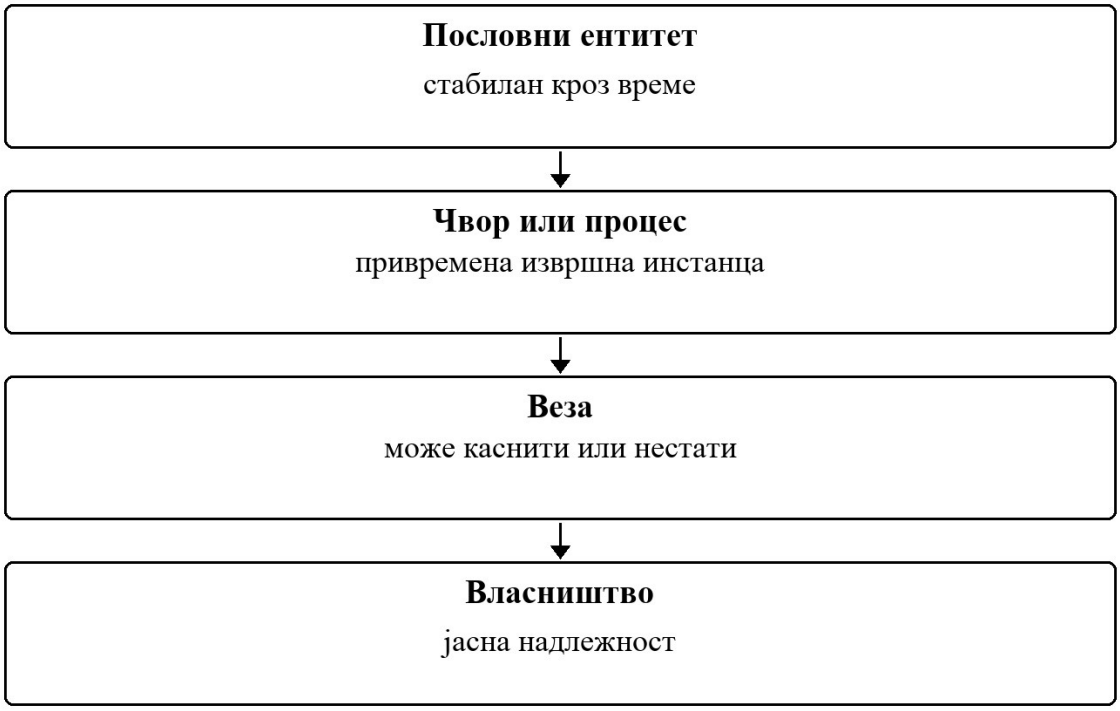
Осам вежби прати ток размишљања потребан за дистрибуирани систем: најпре власништво и границе, затим уговори и видљивост, потом рад са порукама, команде и прекиди везе, а на крају координација, оптерећење и конзистентност. Редослед гради ментални модел дистрибуираног понашања.

- Власништво и границе: пословни ентитети, чворови и неизвесност мреже.
- Уговори и видљивост: поруке, симулатори, корелација и конфигурација.
- Поуздани токови: подаци, команде, послови, поновни покушај и идемпотентност.
- Рад под притиском: прекиди везе, координација, повратни притисак и застарели прикази.

1 Начин размишљања у дистрибуираном систему

Дистрибуирани систем није обична апликација распоређена на више процеса. Компоненте посматрају свет из различитих тренутака, мрежа може каснити или прекинути везу, а потврда операције може стићи након што је локални контекст већ промењен. Зато дизајн почиње јасним власништвом и границама одговорности.

Када се зна која компонента је надлежна за податак или операцију, лакше је објаснити шта се дешава при прекиду везе и ко доноси коначну одлуку. То спречава да више инстанци тихо раде исту ствар или да се нестанак процеса погрешно протумачи као нестанак пословног ентитета.



Слика 1. Власништво, границе и неизвесност пре прве мрежне поруке

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

1.1 Систем не види све у исто време

- Процес, мрежна веза и пословни ентитет нису иста ствар.
- Истек времена значи да одговор није стигао на време, не да се операција сигурно није десила.
- Статус треба да изрази оно што систем зна, а не оно што претпоставља.

Питање за дискусију

Који податак или операција мора имати јасан ауторитет и како систем препознаје да је тај ауторитет привремено недоступан?

1.2 Како се ова тема види у развоју

Тим најпре описује границе система, одговорност сваке компоненте и стање које је за њу ауторитативно. Такав опис касније одређује где се раде провере, где се чува историја и шта се тестира када чвор привремено изгуби везу.

Честе грешке које треба препознати

- Изједначавање прекида везе са трајним кваром.
- Две инстанце које без правила мењају исто стање.
- Пословно правило скривено у комуникационом детаљу.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

1.3 Пример: прекид везе није нестанак стања

Централна компонента престаје да прима хеартбеат од удаљеног чвора. Она зна да потврда није стигла, али не зна да ли је процес престао, мрежа прекинута или је порука изгубљена. Разлика је важна јер погрешна претпоставка може покренути дупли рад или погрешно затворити пословни ток.

- Статус треба да изрази последње поуздано сазнање система.
- Поновно повезивање не сме аутоматски значити да је настала нова пословна инстанца.
- Промена власништва над радом мора имати правило које спречава стари чвор да настави са радом.

Како се сценарио анализира

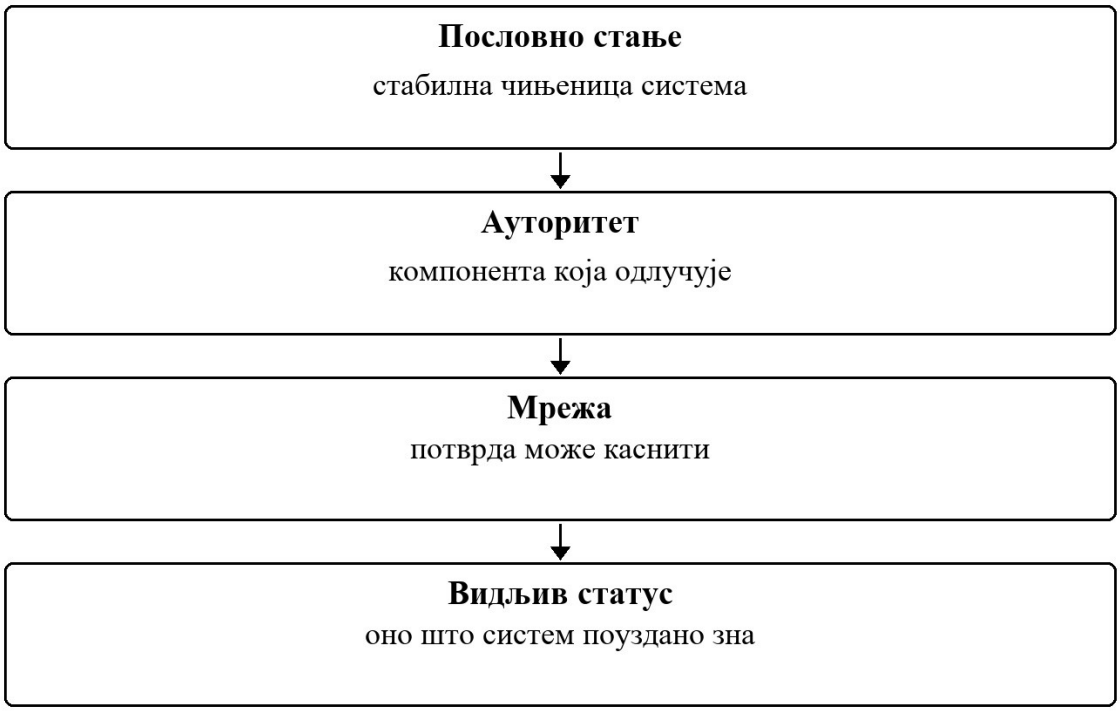
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

1.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

1.5 Шема ауторитета и неизвесности



Слика 2. Шема ауторитета и неизвесности

Процес и веза могу нестати, али пословно стање захтева јасан и стабилан ауторитет.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

1.6 Упоредни преглед елемената тока

Табела 2. Елемент и Улоīа у ток̄у

Елемент	Улога у току	Питање које треба поставити
Пословни ентитет	стабилан кроз време	Шта се дешава када пословни ентитет није доступан или даје застарелу информацију?
Чвор или процес	привремена извршна инстанца	Шта се дешава када чвор или процес није доступан или даје застарелу информацију?
Веза	може каснити или нестати	Шта се дешава када веза није доступан или даје застарелу информацију?
Власништво	јасна надлежност	Шта се дешава када власништво није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

1.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Процес, мрежна веза и пословни ентитет нису иста ствар.
2. Истек времена значи да одговор није стигао на време, не да се операција сигурно није десила.
3. Статус треба да изрази оно што систем зна, а не оно што претпоставља.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Који податак или операција мора имати јасан ауторитет и како систем препознаје да је тај ауторитет привремено недоступан?

1.8 Извршиви пример

Пример раздваја пословни ентитет од процеса који га представља. Када удаљени чвор престане да се јавља, систем зна само колико дуго ћути; не зна да ли је процес пао, мрежа прекинута или је порука изгубљена. Зато статус говори оно што систем зна, а чвор остаје регистрован.

Пример 1. Покрет̄ање демонстрације и тест̄ова

```
cd examples/odp-dotnet/vezba-01-vlasnistvo-neizvesnost
dotnet run --project src/Odp.Vezba01.ConsoleUi
dotnet test Odp.Vezba01.sln
```

Правило у коду

Пример 2. examples/odp-dotnet/vezba-01-vlasnistvo-neizvesnost/src/Odp.Vezba01.Domain/Liveness/NodeLivenessPolicy.cs

```
public NodeLivenessAssessment Evaluate(StationNode node, DateTimeOffset now)
{
    ArgumentNullException.ThrowIfNull(node);

    var silence = SilenceOf(node, now);
    if (silence < _thresholds.SuspectAfter)
```

```
        return new(NodeStatus.Online, NodeLivenessCodes.HeartbeatFresh, silence);
    if (silence < _thresholds.UnreachableAfter)
        return new(NodeStatus.Suspected, NodeLivenessCodes.HeartbeatLate, silence);
    return new(NodeStatus.Unreachable, NodeLivenessCodes.HeartbeatMissing, silence);
}

private static TimeSpan SilenceOf(StationNode node, DateTimeOffset now) =>
    now > node.LastHeartbeatAt ? now - node.LastHeartbeatAt : TimeSpan.Zero;
```

Тест који задаје квар

Пример 3. examples/odp-dotnet/vezba-01-vlasnistvo-neizvesnost/tests/Odp.Vezba01.Tests/Application/MonitorNodesHandlerTests.cs

```
[Test]
public void Assess_WhenNodeStaysSilent_ReportsItUnreachableButKeepsItRegistered()
{
    var clock = new ManualClock(Start);
    var registry = new InMemoryNodeRegistry([new StationNode("node-ns-1", "GS-NOVI-SAD",
Start)]);
    var handler = new MonitorNodesHandler(
        registry,
        new NodeLivenessPolicy(new LivenessThresholds(TimeSpan.FromSeconds(30),
TimeSpan.FromSeconds(90))),
        clock);

    clock.Advance(TimeSpan.FromMinutes(10));
    var view = handler.Assess().Single();

    using (Assert.EnterMultipleScope())
    {
        Assert.That(view.Liveness.Status, Is.EqualTo(NodeStatus.Unreachable));
        Assert.That(view.Liveness.Code, Is.EqualTo(NodeLivenessCodes.HeartbeatMissing));
        Assert.That(registry.Find("node-ns-1"), Is.Not.Null);
    }
}
```

Испис демонстрације

Пример 4. дотнет рун --пројецт src/Odp.Vezba01.ConsoleUi

```
POČETAK [09:00:00]
  node-bg-1 (GS-BEOGRAD): Online | heartbeat_fresh | tišina 0 s
  node-ns-1 (GS-NOVI-SAD): Online | heartbeat_fresh | tišina 0 s
  heartbeat node-ns-1 poslat u 09:00:45 -> heartbeat_accepted
POSLE 45 s (javio se samo node-ns-1) [09:00:45]
  node-bg-1 (GS-BEOGRAD): Suspected | heartbeat_late | tišina 45 s
  node-ns-1 (GS-NOVI-SAD): Online | heartbeat_fresh | tišina 0 s
POSLE 105 s [09:01:45]
  node-bg-1 (GS-BEOGRAD): Unreachable | heartbeat_missing | tišina 105 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
  heartbeat node-ns-1 poslat u 09:00:20 -> heartbeat_out_of_order
STIGAO ZAKASNELI HEARTBEAT ZA node-ns-1 [09:01:45]
  node-bg-1 (GS-BEOGRAD): Unreachable | heartbeat_missing | tišina 105 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
  heartbeat node-bg-1 poslat u 09:01:45 -> heartbeat_accepted
node-bg-1 SE PONOVO JAVIO [09:01:45]
  node-bg-1 (GS-BEOGRAD): Online | heartbeat_fresh | tišina 0 s
  node-ns-1 (GS-NOVI-SAD): Suspected | heartbeat_late | tišina 60 s
```

Шта треба уочити

- Статус зависи само од дужине тишине; разлог тишине систем не зна и не претпоставља.
- Чвор који је `Unreachable` остаје у регистру: нестанак процеса није нестанак ентитета.
- Закаснели хеартбеат добија код `heartbeat_out_of_order` и не враћа познато стање уназад.

Задатак уз пример

Додати статус `Draining` за чвор који је најавио планирано гашење. Такав чвор не сме постати `Unreachable` док траје најављени период. Правило додати у `NodeLivenessPolicy`, увести нов код и покрити га тестом.

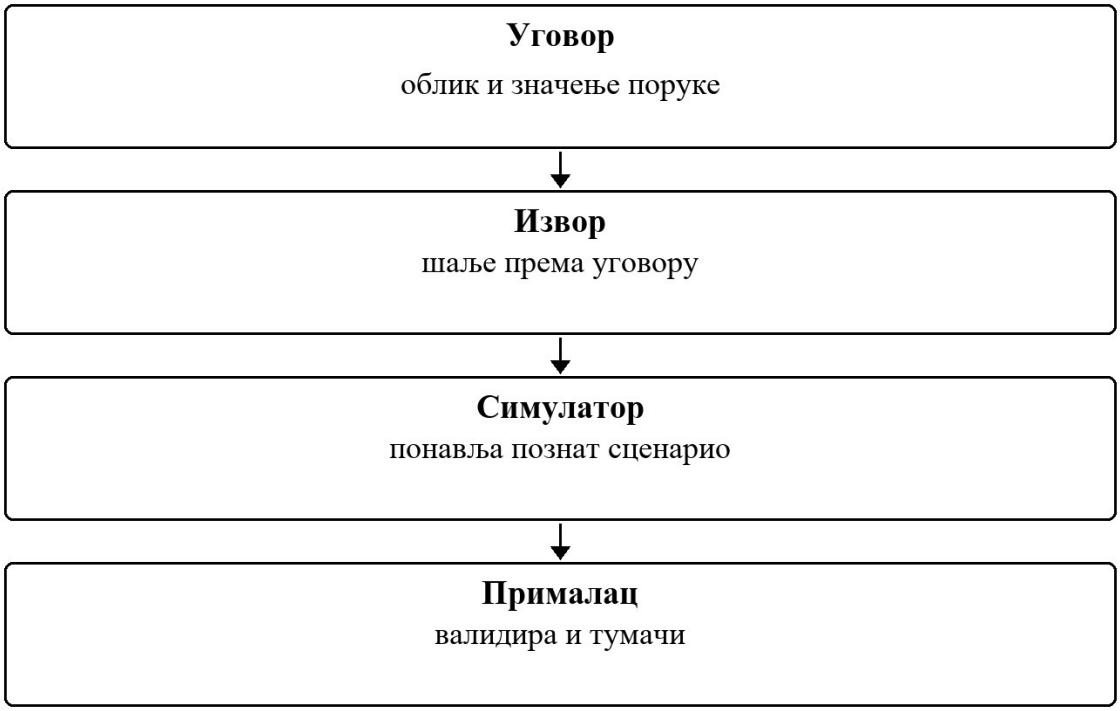
Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Начин размишљања у дистрибуираном систему". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

2 Уговори, симулатори и поновљивост

Компоненте могу да сарађују само ако деле уговор о облику поруке и очекиваном понашању. Уговор није споредна документација: он је граница која омогућава независну промену и даје јасан одговор на питање да ли порука има значење за примаоца.

Симулатор даје тиму поновљив начин да провери токове без ослањања на реалну опрему, нестабилну мрежу или случајне улазе. Поновљивост је услов да се грешка разуме и да поправка заиста буде доказана.



Слика 3. Заједнички језик компоненти и безбедан начин учења из грешке

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

2.1 Порука мора да има заједничко значење

- Објављена верзија уговора чува очекивања постојећих корисника.
- Валидација одбија поруку коју систем не разуме, уместо да јој нагађа значење.
- Симулатор одваја стварни проблем система од случајности спољног света.

Питање за дискусију

Коју поруку бисте најпре претворили у стабилан уговор и који сценарио симулатора би најбоље открио неспоразум између пошиљаоца и примаоца?

2.2 Како се ова тема види у развоју

Тим записује мали скуп порука и сценарија који представљају кључне токове. Затим покреће исти сценарио више пута, са познатим улазима, и проверава да сваки пут добија исти релевантан резултат.

Честе грешке које треба препознати

- Промена поруке без верзије и без размишљања о примаоцима.
- Симулатор који садржи пословну логику централног система.
- Тест који зависи од случајног времена или спољне услуге.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

2.3 Пример: прималац не разуме нову поруку

Пошиљалац проширује поруку новим пољем и сматра да промена нема последице. Старији прималац, међутим, другачије тумачи вредност или одбацује поруку. Уговор чува обе стране тако што облик, верзију и правила компатибилности чини експлицитним пре него што порука стигне до продукционог тока.

- Верзија говори примаоцу како да тумачи поруку.
- Симулатор може поновити исти ток са старим и новим примаоцем.
- Нејасна или непозната порука добија контролисан исход, а не произвољно тумачење.

Како се сценарио анализира

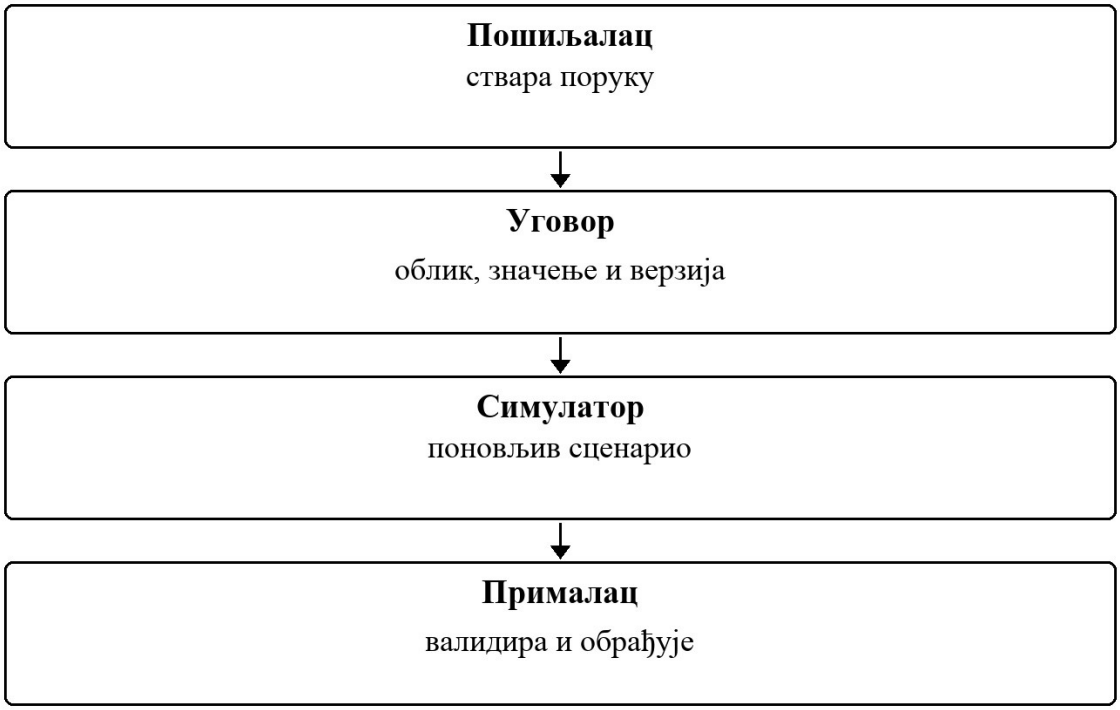
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

2.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

2.5 Шема уговора између компоненти



Слика 4. Шема уговора између компоненти

Стабилан уговор омогућава компонентама да се мењају без нагађања о значењу поруке.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

2.6 Упоредни преглед елемената тока

Табела 3. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Уговор	облик и значење поруке	Шта се дешава када уговор није доступан или даје застарелу информацију?
Извор	шаље према уговору	Шта се дешава када извор није доступан или даје застарелу информацију?
Симулатор	понавља познат сценарио	Шта се дешава када симулатор није доступан или даје застарелу информацију?
Прималац	валидира и тумачи	Шта се дешава када прималац није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

2.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Објављена верзија уговора чува очекивања постојећих корисника.
2. Валидација одбија поруку коју систем не разуме, уместо да јој нагађа значење.
3. Симулатор одваја стварни проблем система од случајности спољног света.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Коју поруку бисте најпре претворили у стабилан уговор и који сценарио симулатора би најбоље открио неспоразум између пошиљаоца и примаоца?

2.8 Извршиви пример

Пример показује две ствари које омогућавају да се компоненте развијају независно: порука има експлицитан уговор који се проверава на улазу, а удаљена станица се замењује симулатором чији је ток у потпуности одређен сценаријом. Исти сценарио увек даје исте поруке, па се и грешка може поновити.

Пример 5. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-02-ugovori-simulator
dotnet run --project src/Odp.Vezba02.ConsoleUi
dotnet test Odp.Vezba02.sln
```

Правило у коду

Пример 6. examples/odp-dotnet/vezba-02-ugovori-simulator/src/Odp.Vezba02.Domain/Contracts/TelemetryContractValidator.cs

```
public Result Validate(TelemetryMessage message)
{
    ArgumentNullException.ThrowIfNull(message);

    if (message.ContractVersion != TelemetryContract.Version)
        return Result.Fail(ContractErrorCodes.UnsupportedVersion);
    if (string.IsNullOrWhiteSpace(message.StationId))
        return Result.Fail(ContractErrorCodes.StationIdRequired);
    if (message.Sequence <= 0)
        return Result.Fail(ContractErrorCodes.SequenceMustBePositive);
    if (message.SignalStrengthDbm is < TelemetryContract.MinSignalDbm or >
```

```
TelemetryContract.MaxSignalDbm)
        return Result.Fail(ContractErrorCodes.SignalOutOfRange);

    return Result.Ok();
}
```

Тест који задаје квар

Пример 7. examples/odp-dotnet/vezba-02-ugovori-simulator/tests/Odp.Vezba02.Tests/Infrastructure/ScriptedStationSimulatorTests.cs

```
[Test]
public void Emit_WithTheSameScenario_ProducesTheSameMessages()
{
    var first = new ScriptedStationSimulator(Scenario).Emit(20);
    var second = new ScriptedStationSimulator(Scenario).Emit(20);

    Assert.That(second, Is.EqualTo(first));
}
```

Испис демонстрације

Пример 8. дотнет рун --пројецт src/Odp.Vezba02.ConsoleUi

```
Poruke simulatora:
  #1 09:00:00  -83.0 dBm
  #2 09:00:05 -105.0 dBm
  #3 09:00:10 -105.0 dBm
  #4 09:00:15    5.0 dBm
  #5 09:00:20 -104.0 dBm
  #6 09:00:25 -100.0 dBm
  #7 09:00:30  -81.0 dBm
  #8 09:00:35    5.0 dBm

Prihvaćeno: 6, odbijeno: 2
  #4 -> signal_out_of_range
  #8 -> signal_out_of_range

Ponovljeno pokretanje daje isti ishod: True
```

Шта треба уочити

- Уговор се проверава на улазу, а свако одбијање има стабилан код.
- Исти сценарио (исти `Seed`) даје исте поруке, па се и грешка може поновити.
- Симулатор је адаптер иза порта `IStationSimulator`; правила не знају да станица није права.

Задатак уз пример

Додати у сценарио кашњење поруке (`DelayEvery`), тако да свака Н-та порука носи `MeasuredAt` старији од претходне. Увести правило уговора које такву поруку одбија новим кодом и покрити га тестом.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Уговори, симулатори и поновљивост". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

П1 Основни ентитети, уговори и симулатор

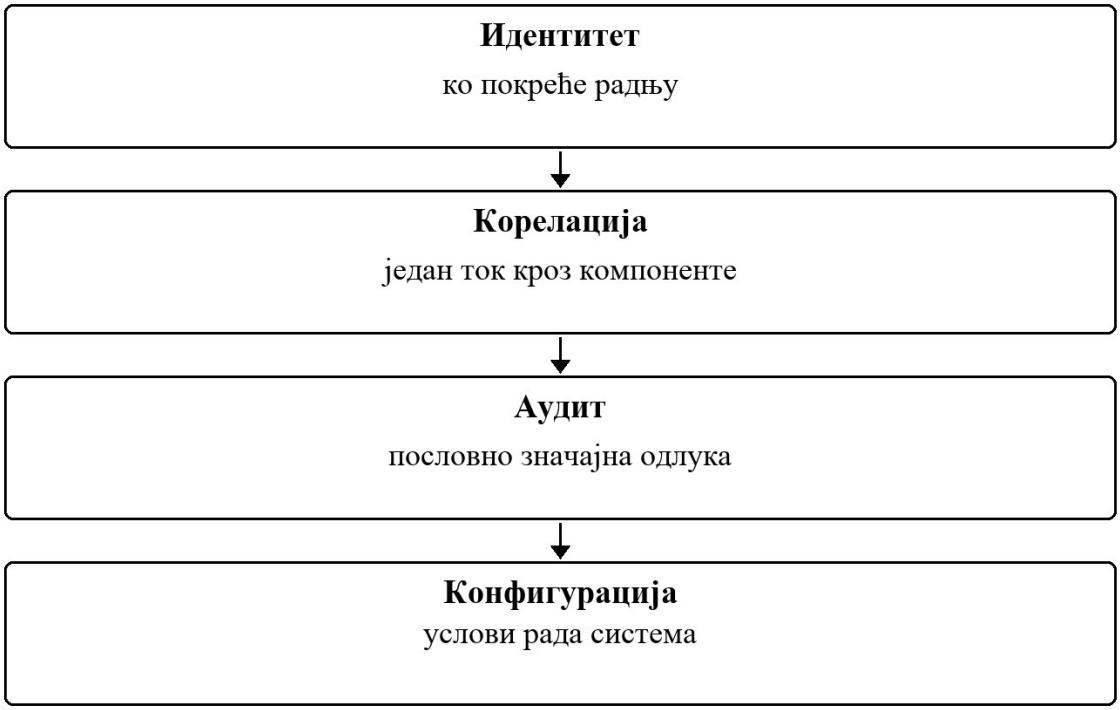
Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.

3 Идентитет, аудит, могућност посматрања система и конфигурација

Дистрибуирани систем мора моћи да одговори на четири питања: ко је покренуо радњу, шта се догодило, кроз које компоненте је ток прошао и под којим условима је радио. Идентитет, аудит, могућност посматрања система и конфигурација дају одговоре који се не смеју измишљати изнова у свакој функционалности.

Када се ток поквари, појединачни логови нису довољни. Заједнички корелациони идентификатор и јасан аудит траг омогућавају да тим састави причу о операцији без нагађања и без излагања осетљивих података.



Слика 5. Попречне могућности које свака компонента користи на исти начин

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

3.1 Видљивост мора бити уграђена

- Аудит чува одлуке које су важне за одговорност и истрагу.
- Логови и трагови чувају технички контекст, али не тајне.
- Конфигурација има валидно значење, власника и начин промене.

Питање за дискусију

Како бисте реконструисали једну неуспелу операцију која је прошла кроз Програмски интерфејс, ред порука и воркер, а да не читате неограничену количину логова?

3.2 Како се ова тема види у развоју

Тим бира неколико операција које мора умети да прати од захтева до исхода. Свака компонента преноси заједнички контекст, а тестови проверавају да промена конфигурације или прекид тока оставља довољно информација за разумевање.

Честе грешке које треба препознати

- Лог који открива креденцијале или личне податке.
- Корелациони идентификатор креиран у свакој компоненти уместо пренет.
- Конфигурација прихваћена без провере опсега и утицаја.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

3.3 Пример: захтев пролази кроз три компоненте

Корисник покреће операцију кроз Програмски интерфејс, Програмски интерфејс објављује поруку, а воркер завршава обраду. Ако свака компонента бележи свој неповезан идентификатор, тим не може брзо да одговори да ли је захтев прихваћен, обрађен или изгубљен између корака. Заједнички контекст претвара три засебна записа у једну проверљиву причу.

- Корелациони идентификатор настаје на граници тока и преноси се даље.
- Аудит чува одлуке које утичу на одговорност, док лог чува технички контекст.
- Конфигурација одређује очекивано понашање, као што су рокови и прагови, и мора бити проверљива.

Како се сценарио анализира

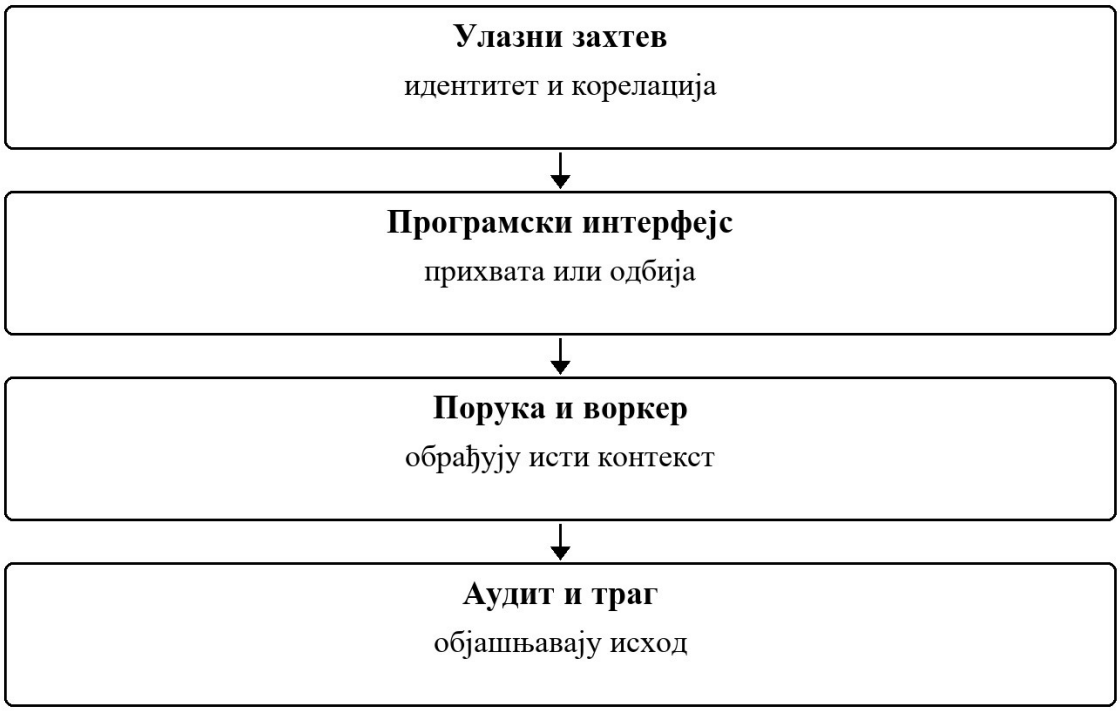
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

3.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

3.5 Шема следљивог дистрибуираног тока



Слика 6. Шема следљивоī дисшрибуираноī шока

Један контекст пролази кроз све компоненте, па се неуспех може објаснити без нагађања.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преносе и сценарио отказа који може променити ток.

3.6 Упоредни преглед елемената тока

Табела 4. Елемен̄ш и Улоīа у ш̄оку

Елемент	Улога у току	Питање које треба поставити
Идентитет	ко покреће радњу	Шта се дешава када идентитет није доступан или даје застарелу информацију?
Корелација	један ток кроз компоненте	Шта се дешава када корелација није доступан или даје застарелу информацију?
Аудит	пословно значајна одлука	Шта се дешава када аудит није доступан или даје застарелу информацију?
Конфигурација	услови рада система	Шта се дешава када конфигурација није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

3.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Аудит чува одлуке које су важне за одговорност и истрагу.

2. Логови и трагови чувају технички контекст, али не тајне.

3. Конфигурација има валидно значење, власника и начин промене.

4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Како бисте реконструисали једну неуспелу операцију која је прошла кроз Програмски интерфејс, ред порука и воркер, а да не читате неограничену количину логова?

3.8 Извршиви пример

Пример прати једну операцију кроз две компоненте. Заказивање контакта са станицом добија идентификатор операције (`CorrelationId`) и идентитет актера; исти идентификатор стиже до станице и до ревизијског трага, па се ток може реконструисати и када делови система воде одвојене логове. Ограничење трајања контакта долази из конфигурације која се проверава пре покретања.

Пример 9. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-03-identitet-audit-konfiguracija
dotnet run --project src/Odp.Vezba03.ConsoleUi
dotnet test Odp.Vezba03.sln
```

Правило у коду

Пример 10. examples/odp-dotnet/vezba-03-identitet-audit-konfiguracija/src/Odp.Vezba03.Application/Contacts/ScheduleContactHandler.cs

```
public ScheduleContactResult Schedule(ContactRequest request, OperationContext context)
{
    ArgumentNullException.ThrowIfNull(request);
    ArgumentNullException.ThrowIfNull(context);

    var code = Decide(request, context);

    auditLog.Record(new AuditEntry(
        clock.UtcNow,
        context.CorrelationId,
        context.ActorId,
        Action,
        request.StationId,
        code));

    return new ScheduleContactResult(code == ContactCodes.Scheduled, code,
context.CorrelationId);
}
```

Тест који задаје квар

Пример 11. examples/odp-dotnet/vezba-03-identitet-audit-konfiguracija/tests/Odp.Vezba03.Tests/Application/ScheduleContactHandlerTests.cs

```
[Test]
public void Schedule_CarriesTheSameCorrelationIdToTheStationAndTheAuditTrail()
{
    var result = _handler.Schedule(Request, Context);

    using (Assert.EnterMultipleScope())
    {
        Assert.That(result, Is.EqualTo(new ScheduleContactResult(true,
ContactCodes.Scheduled, "op-42")));
        Assert.That(_stationGateway.Calls.Single().CorrelationId, Is.EqualTo("op-42"));
        Assert.That(
            _auditLog.Entries.Single(),
            Is.EqualTo(new AuditEntry(
                Now,
                "op-42",
                "ana",
                "contact.schedule",
                "GS-NOVI-SAD",
                ContactCodes.Scheduled)));
```

```
    }
}
```

Испис демонстрације

Пример 12. дотнет рун --пројецт src/Одп.Vezba03.ConsoleUi

```
op-0001: GS-NOVI-SAD, 10 min -> contact_scheduled
op-0002: GS-NOVI-SAD, 40 min -> duration_too_long
op-0003: GS-BEOGRAD, 10 min -> station_unavailable

Log stanice:
  op-0001 | GS-NOVI-SAD | reserved=True
  op-0003 | GS-BEOGRAD | reserved=False

Revizijski trag:
  op-0001 | ana | contact.schedule | GS-NOVI-SAD | contact_scheduled
  op-0002 | marko | contact.schedule | GS-NOVI-SAD | duration_too_long
  op-0003 | jelena | contact.schedule | GS-BEOGRAD | station_unavailable
```

Шта треба уочити

- Исти `CorrelationId` види се у логу станице и у ревизијском трагу центра.
- Ревизијски запис настаје и када је захтев одбијен пре позива станице.
- Неисправна конфигурација зауставља покретање са кодом грешке.

Задатак уз пример

Додати у `OperationContext` идентификатор позивајуће компоненте (`Source`) и уписати га у ревизијски запис. Проширити тест тако да доказује да запис садржи и извор операције.

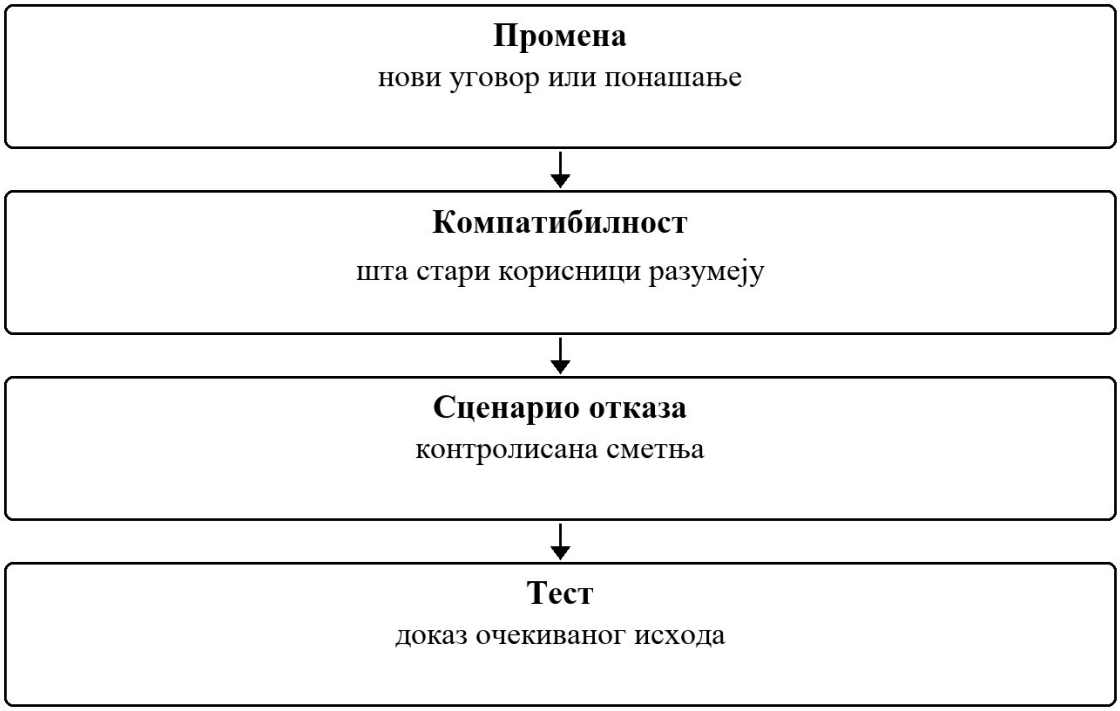
Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Идентитет, аудит, могућност посматрања система и конфигурација". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

4 Верзионисање и полазећи од отказа тестирање

У дистрибуираном систему нормалан сценарио никада није цела прича. Порука може да стигне два пута, ван редоследа или након истека рока. Верзије уговора и контролисани отказ сценарији омогућавају тиму да те ситуације проучи док су безбедне и разумљиве.

Полазећи од отказа приступ не предвиђа сваки могући квар. Он подстиче тим да за важну операцију постави права питања: шта ако одговор касни, шта ако се захтев понови и шта ако компонента нестане између две радње?



Слика 7. Развој који рачуна на кашњење, дупликате и прекиде

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

4.1 Неизвесност је део уговора

- Компатибилност се процењује према примаоцима поруке, не само према коду пошиљаоца.
- Кашњење и дупликат су нормални улази које компонента мора да објасни.
- Тест треба да провери видљив исход, не само да метода не баца изузетак.

Питање за дискусију

За коју операцију би касно пристигла порука била опаснија од потпуно изгубљене поруке и зашто?

4.2 Како се ова тема види у развоју

За најважније токове тим формулише неколико малих отказ експеримената и покреће их у истом окружењу. Резултат улази у тестну базу и у опис одлуке, па следећа промена не мора поново да открива иста ограничења.

Честе грешке које треба препознати

- Претпоставка да се порука испоручује тачно једном.
- Некомпатибилна промена означена као безопасна јер локални тестови пролазе.

- Тестирање прекида тек након интеграције свих компоненти.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

4.3 Пример: потврда стиже након истека рока

Компонента чека одговор до истека рока и покреће безбедан опоравак. Затим стигне стара потврда из првог покушаја. Ако систем нема стабилан идентитет операције и правило за редослед, та потврда може преписати новије стање. Сценарио отказа зато није изузетак који се само ухвати, већ део уговора о понашању.

- Истек времена значи да одговор није стигао на време, не да се радња сигурно није извршила.
- Дупликат и касна порука морају се раздвојити од нове легитимне операције.
- Поновљив симулатор омогућава да се исти редослед грешке провери након сваке измене.

Како се сценарио анализира

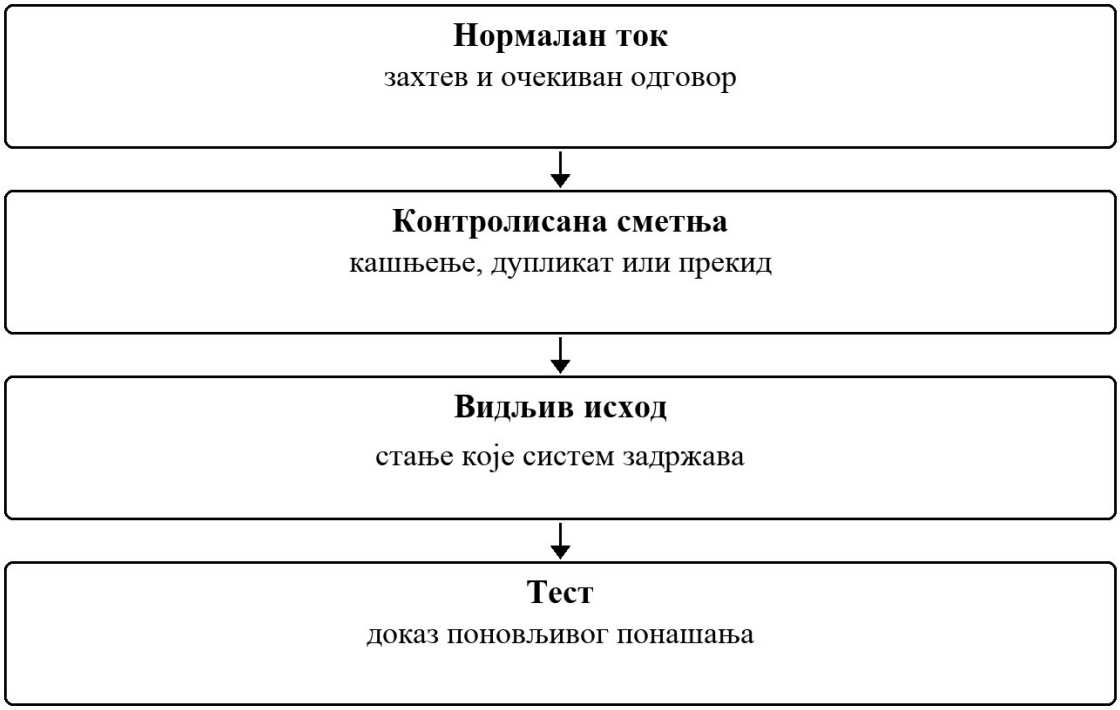
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

4.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

4.5 Шема полазећи од отказа провере



Слика 8. Шема полазећи од отказа провере

Циљ није претпоставити савршен транспорт, већ јасно дефинисати понашање када транспорт није савршен.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

4.6 Упоредни преглед елемената тока

Табела 5. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Промена	нови уговор или понашање	Шта се дешава када промена није доступан или даје застарелу информацију?
Компатибилност	шта стари корисници разумеју	Шта се дешава када компатибилност није доступан или даје застарелу информацију?
Сценарио отказа	контролисана сметња	Шта се дешава када сценарио отказа није доступан или даје застарелу информацију?
Тест	доказ очекиваног исхода	Шта се дешава када тест није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

4.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Компатибилност се процењује према примаоцима поруке, не само према коду пошиљаоца.
2. Кашњење и дупликат су нормални улази које компонента мора да објасни.
3. Тест треба да провери видљив исход, не само да метода не баца изузетак.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

За коју операцију би касно пристигла порука била опаснија од потпуно изгубљене поруке и зашто?

4.8 Извршиви пример

Пример полази од онога што мрежа сигурно ради: иста порука може стићи два пута, може закаснити и може бити послата новијом верзијом уговора. Пријемна страна зато за сваку поруку доноси експлицитну одлуку, а тестови најпре покривају неуспешне путеве.

Пример 13. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-04-verzionisanje-failure-first
dotnet run --project src/Odp.Vezba04.ConsoleUi
dotnet test Odp.Vezba04.sln
```

Правило у коду

Пример 14. examples/odp-dotnet/vezba-04-verzionisanje-failure-first/src/Odp.Vezba04.Domain/Messaging/InboxPolicy.cs

```
public InboxDecision Decide(MessageEnvelope envelope, bool alreadyProcessed,
DateTimeOffset now)
{
    ArgumentNullException.ThrowIfNull(envelope);

    if (envelope.Version.Major != rules.SupportedMajor)
        return new(false, InboxCodes.UnsupportedMajorVersion);
    if (alreadyProcessed)
        return new(false, InboxCodes.Duplicate);
```

```
    if (now - envelope.SentAt > rules.MaxAge)
        return new(false, InboxCodes.TooOld);

    return new(true, InboxCodes.Accepted);
}
```

Тест који задаје квар

Пример 15. examples/odp-dotnet/vezba-04-verzionisanje-failure-first/tests/Odp.Vezba04.Tests/Application/ReceiveMessageHandlerTests.cs

```
[Test]
public void Receive_WhenTheSameMessageIsDeliveredTwice_ProcessesItOnce()
{
    var envelope = Envelope("m-1", 1, Start);

    var first = _handler.Receive(envelope);
    var second = _handler.Receive(envelope);

    using (Assert.EnterMultipleScope())
    {
        Assert.That(first.Code, Is.EqualTo(InboxCodes.Accepted));
        Assert.That(second.Code, Is.EqualTo(InboxCodes.Duplicate));
        Assert.That(_processor.ProcessedPayloads, Has.Count.EqualTo(1));
    }
}
```

Испис демонстрације

Пример 16. дотнет рун --пројецт src/Odp.Vezba04.ConsoleUi

```
m-1 v1.0 (prva isporuka) -> message_accepted
m-1 v1.0 (ponovljena isporuka iste poruke) -> message_duplicate
m-2 v1.3 (novija minor verzija) -> message_accepted
m-3 v2.0 (druga major verzija) -> message_major_version_unsupported
m-4 v1.0 (poruka koja je putovala 12 minuta) -> message_too_old

Poslovni efekat se dogodio 2 puta:
  telemetrija m-1
  telemetrija m-2
```

Шта треба уочити

- Дупликат, закаснела порука и друга мајор верзија су очекивани улази са сопственим кодом.
- Пословни ефекат се дешава једном, колико год пута да иста порука стигне.
- Новија минор верзија се прихвата; друга мајор верзија се одбија.

Задатак уз пример

Додати правило да се порука са минор верзијом већом од познате прихвата, али се исход означава новим кодом `message_accepted_newer_minor`, како би тим знао да уговор треба прегледати. Покрити гранични случај тестом.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Верзионисање и полазећи од отказа тестирање". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

П2 Идентитет, аудит, могућност посматрања система и конфигурација

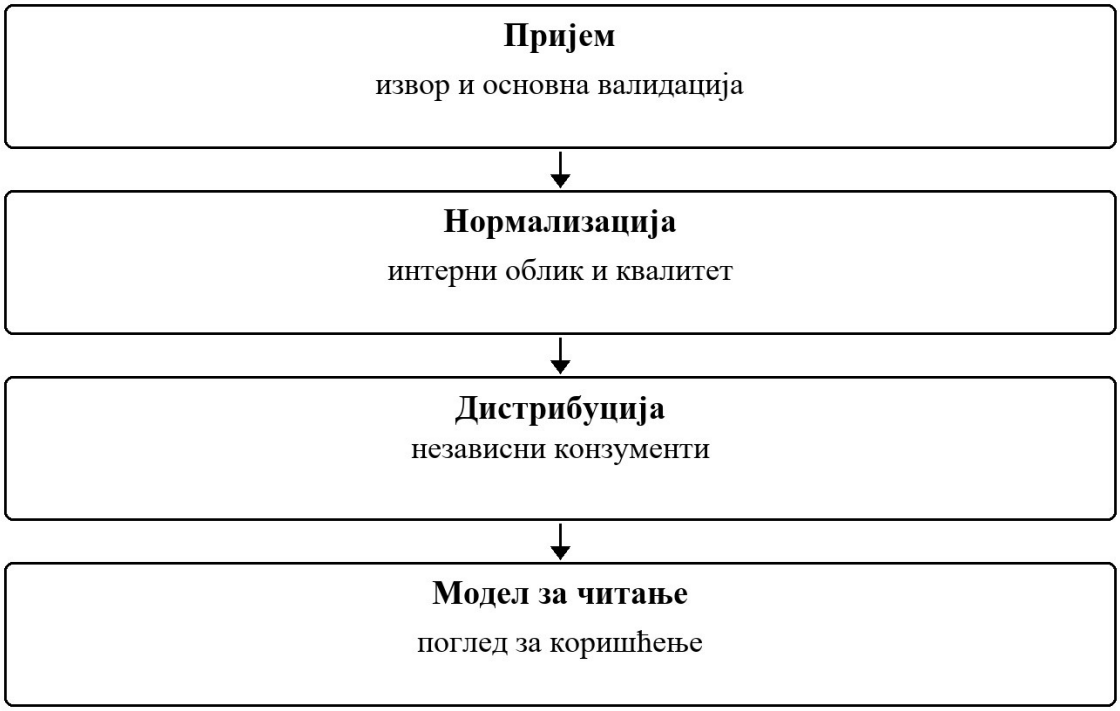
Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.

5 Ток података: пријем, валидација, роутинг и читање

Податак који стигне преко мреже још није поуздана пословна информација. Потребно је утврдити порекло, валидност, квалитет и начин на који ће различити делови система добити свој поглед на њега. Раздвајање ових одговорности чува јасноћу када се појаве кашњења или дупликати.

Ток података обично служи више корисника са различитим потребама. Роутинг одваја њихову обраду, а модели за читање прилагођавају податак читању. Цена тог избора је да приказ може каснити, што систем треба отворено да покаже.



Слика 9. Од поруке са мреже до информације којој корисник може да верује

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

5.1 Порекло и квалитет путују са податком

- Стабилан идентитет поруке омогућава да се препозна њено понављање.
- Време настанка и време пријема имају различито значење.
- Модел за читање показује корисну слику, али мора знати границу своје свежине.

Питање за дискусију

Који податак у вашем систему може каснити, а ипак остати користан? Како корисник треба да види ту ограду?

5.2 Како се ова тема види у развоју

Тим описује који слој сме да одбије поруку, који јој додељује квалитет и који је чува за касније читање. Та подела олакшава тестирање јер свака одлука има свој улаз, видљив исход и разлог у евиденцији.

Честе грешке које треба препознати

- Тихо одбацивање поруке без разлога и трага.
- Мешање времена извора и пријема.

- Модел за читање који дуплира пословни ефекат при поновљеној поруци.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

5.3 Пример: порука стиже два пута и касно

Телеметријска порука може стићи два пута, а њено време настанка може бити старије од времена пријема. Ако складиште само додаје сваки запис, приказ последње вредности може постати погрешан или претрпан дупликатима. Ток података зато чува идентитет поруке, порекло и оба времена док не формира модел за читање за корисника.

- Пријем проверава извор и минимални облик поруке.
- Нормализација чува квалитет и разлику између времена настанка и пријема.
- Модел за читање отворено приказује свежину информације и не претвара дупликат у нови пословни догађај.

Како се сценарио анализира

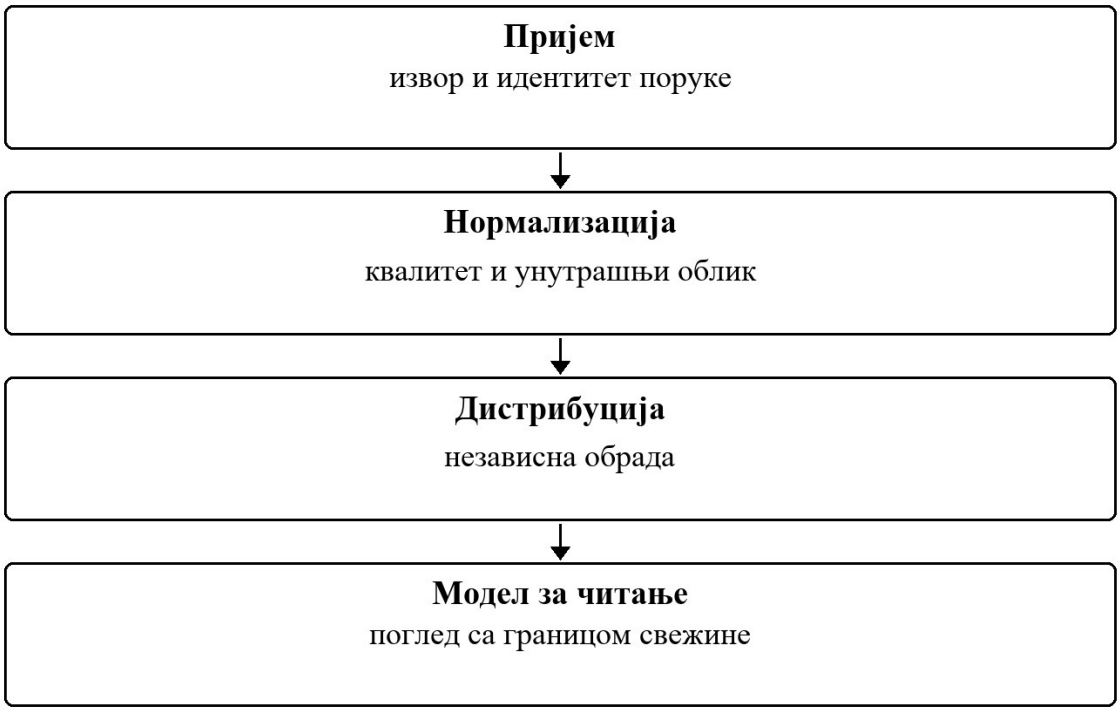
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

5.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

5.5 Шема тока података



Слика 10. Шема тока података

Податак постаје поуздан тек када су познати његово порекло, квалитет и границе приказа.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

5.6 Упоредни преглед елемената тока

Табела 6. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Пријем	извор и основна валидација	Шта се дешава када пријем није доступан или даје застарелу информацију?
Нормализација	интерни облик и квалитет	Шта се дешава када нормализација није доступан или даје застарелу информацију?
Дистрибуција	независни конзументи	Шта се дешава када дистрибуција није доступан или даје застарелу информацију?
Модел за читање	поглед за коришћење	Шта се дешава када модел за читање није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

5.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Стабилан идентитет поруке омогућава да се препозна њено понављање.
2. Време настанка и време пријема имају различито значење.
3. Модел за читање показује корисну слику, али мора знати границу своје свежине.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Који податак у вашем систему може каснити, а ипак остати користан? Како корисник треба да види ту ограду?

5.8 Извршиви пример

Пример раздваја запис од приказа. Свако прихваћено мерење улази у запис редом којим је стигло, а приказ станице се изводи из записа и памти само последње мерење. Мерење које закасни остаје у запису, али не враћа приказ уназад; приказ који дуго није освежен се означава као застарео уместо да изгледа као тренутно стање.

Пример 17. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-05-tok-podataka-read-model
dotnet run --project src/Odp.Vezba05.ConsoleUi
dotnet test Odp.Vezba05.sln
```

Правило у коду

Пример 18. examples/odp-dotnet/vezba-05-tok-podataka-read-model/src/Odp.Vezba05.Domain/ReadModels/StationViewProjector.cs

```
public (StationView View, bool Changed) Apply(StationView? current, TelemetryReading
reading)
{
    ArgumentNullException.ThrowIfNull(reading);

    if (current is not null && reading.Sequence <= current.LastSequence)
```

```
        return (current, false);

    return (
        new StationView(
            reading.StationId,
            reading.Sequence,
            reading.MeasuredAt,
            reading.SignalStrengthDbm),
        true);
}
```

Тест који задаје квар

Пример 19. examples/odp-dotnet/vezba-05-tok-podataka-read-model/tests/Odp.Vezba05.Tests/Application/IngestTelemetryHandlerTests.cs

```
[Test]
public void Ingest_WhenOlderReadingArrivesLate_KeepsItInTheLogButNotInTheView()
{
    _handler.Ingest(Reading(3, -88));

    var outcome = _handler.Ingest(Reading(2, -97));

    using (Assert.EnterMultipleScope())
    {
        Assert.That(outcome, Is.EqualTo(new IngestOutcome(true, TelemetryCodes.Stored,
ViewChanged: false)));
        Assert.That(_log.ForStation("GS-NOVI-SAD"), Has.Count.EqualTo(2));
        Assert.That(_views.Find("GS-NOVI-SAD")!.LastSequence, Is.EqualTo(3));
    }
}
```

Испис демонстрације

Пример 20. дотнет рун --пројецт src/Odp.Vezba05.ConsoleUi

```
merenje #1 -> reading_stored, prikaz promenjen: True
merenje #3 -> reading_stored, prikaz promenjen: True
merenje #2 -> reading_stored, prikaz promenjen: False
merenje #0 -> sequence_must_be_positive, prikaz promenjen: False
[posle prijema] #3 -88 dBm, staro 5 s, zastarelo: False
[dva minuta bez novih merenja] #3 -88 dBm, staro 125 s, zastarelo: True

Zapis sadrži 3 merenja, prikaz samo poslednje.
```

Шта треба уочити

- Запис чува сва прихваћена мерења; приказ чува само последње.
- Мерење које закасни остаје у запису, али не мења приказ.
- Приказ који дуго није освежен остаје видљив, али је означен као застарео.

Задатак уз пример

Додати случај употребе `RebuildStationView` који брише приказ станице и поново га изводи из записа. Тестом показати да је резултат исти без обзира на редослед којим су мерења првобитно стигла.

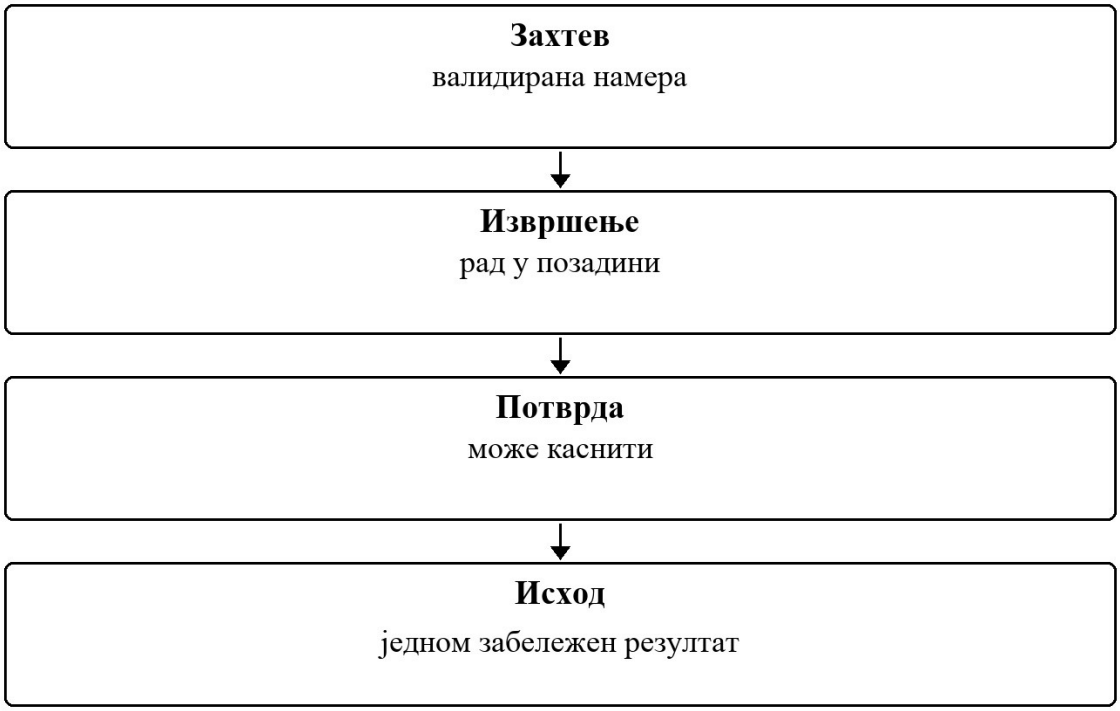
Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Ток података: пријем, валидација, роутинг и читање". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

6 Команде, послови, поновни покушај и идемпотентност

Команда је захтев за промену света, а не обичан позив методе. Након слања, систем може чекати, понављати покушај или добити касну потврду. Зато се животни циклус команде моделује експлицитно и разликује од локалне намере корисника.

Поновни покушај повећава вероватноћу успеха, али без идемпотентности може поновити пословни ефекат. Леасе и ограничено власништво над послом спречавају да два воркера истовремено доврше исту радњу након неусаглашеног погледа на стање.



Слика 11. Како систем напредује када потврда није тренутна ни сигурна

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

6.1 Поновни покушај не сме мењати значење радње

- Статус команде описује знање система, не нагађање о удаљеном свету.
- Идемпотенци кључ везује поновљене покушаје за исти пословни резултат.
- Леасе има рок и штити систем од закашњелог рада претходног власника.

Питање за дискусију

Која би последица дуплог извршења била неприхватљива и који идентитет операције спречава такав исход?

6.2 Како се ова тема види у развоју

Тим раздваја пријем захтева, обраду у позадини и приказ исхода. Затим тестира прекид између њих: поновљени захтев, пад воркера и касна потврда не смеју произвести нову пословну радњу нити поништити новије стање.

Честе грешке које треба препознати

- Поновни покушај без границе и без разлога за престанак.

- Две обраде које обе верују да су власници посла.
- Касна потврда која преписује завршено стање.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

6.3 Пример: поновљен захтев за исту радњу

Клијент не добија одговор на време и шаље исти захтев поново. Систем жели да буде отпоран на прекид, па други захтев не сме аутоматски сматрати новом намером. Идемпотенци кључ повезује поновљене покушаје са истим пословним резултатом, док статус животног циклуса јасно одваја прихватање, обраду, чекање и завршетак.

- Поновни покушај користи исти идентитет операције када намера корисника није промењена.
- Воркер који преузима посао има ограничено време власништва и не може завршити рад након истека.
- Касна потврда не сме преписати новије, завршено стање.

Како се сценарио анализира

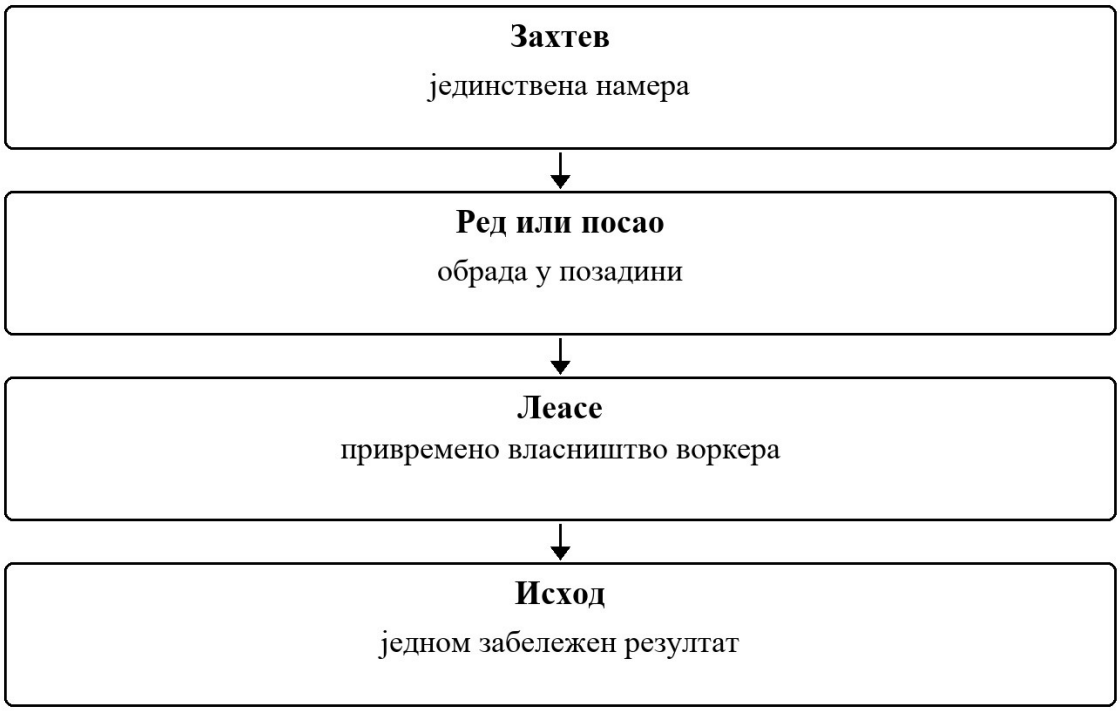
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

6.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

6.5 Шема поуздане команде



Слика 12. Шема поуздане команде

Поновни покушај повећава шансу да се радња заврши, док идемпотентност чува њено значење.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

6.6 Упоредни преглед елемената тока

Табела 7. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Захтев	валидирана намера	Шта се дешава када захтев није доступан или даје застарелу информацију?
Извршење	рад у позадини	Шта се дешава када извршење није доступан или даје застарелу информацију?
Потврда	може каснити	Шта се дешава када потврда није доступан или даје застарелу информацију?
Исход	једном забележен резултат	Шта се дешава када исход није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

6.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Статус команде описује знање система, не нагађање о удаљеном свету.
2. Идемпотенци кључ везује поновљене покушаје за исти пословни резултат.
3. Леасе има рок и штити систем од закашњелог рада претходног власника.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Која би последица дуплог извршења била неприхватљива и који идентитет операције спречава такав исход?

6.8 Извршиви пример

Пример води команду ка удаљеном уређају кроз неизвесност. Слање нема повратну вредност: потврда стиже одвојено, касни или не стигне. Зато команда има стабилан идентификатор, поновно слање носи исти идентификатор, уређај га извршава једном, а потврда која стигне после истека и даље исправља стање.

Пример 21. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-06-komande-idempotentnost
dotnet run --project src/Odp.Vezba06.ConsoleUi
dotnet test Odp.Vezba06.sln
```

Правило у коду

Пример 22. examples/odp-dotnet/vezba-06-komande-idempotentnost/src/Odp.Vezba06.Domain/Commands/CommandLifecycle.cs

```
public (DeviceCommand Command, string Code) Acknowledge(DeviceCommand command) =>
    command.State switch
    {
        CommandState.Sent =>
            (command with { State = CommandState.Acknowledged },
CommandCodes.Acknowledged),
        CommandState.TimedOut =>
```

```
            (command with { State = CommandState.Acknowledged },
CommandCodes.LateAckReconciled),
        _ => (command, CommandCodes.AckDuplicate)
    };
```

Тест који задаје квар

Пример 23. examples/odp-dotnet/vezba-06-komande-idempotentnost/tests/Odp.Vezba06.Tests/Application/CommandFlowTests.cs

```
[Test]
public void Run_WhenAckIsMissing_ResendsTheSameCommandAndTheDeviceExecutesItOnce()
{
    _dispatch.Dispatch("cmd-1", "antena-1", "rotate:120");
    _clock.Advance(TimeSpan.FromSeconds(12));

    var outcomes = _retry.Run();

    using (Assert.EnterMultipleScope())
    {
        Assert.That(outcomes, Is.EqualTo(new[] { new RetryOutcome("cmd-1",
CommandCodes.Retried) }));
        Assert.That(_device.Deliveries, Is.EqualTo(2));
        Assert.That(_device.ExecutedCommandIds, Is.EqualTo(new[] { "cmd-1" }));
    }
}
```

Испис демонстрације

Пример 24. дотнет рун --пројецт src/Odp.Vezba06.ConsoleUi

```
operator šalje komandu -> command_sent (pokušaj 1)
operator ponavlja isti zahtev -> command_already_dispatched (pokušaj 1)
potvrda nije stigla 12 s -> cmd-7: command_retried
potvrda nije stigla još 12 s -> cmd-7: attempts_exhausted
zakasnela potvrda -> late_ack_reconciled
ista potvrda ponovo -> ack_duplicate

Uređaj je primio 2 isporuke, a izvršio 1 komandu.
```

Шта треба уочити

- Поновљени захтев са истим `CommandId` не шаље команду други пут.
- После истека се шаље иста команда; уређај је прима два пута, а извршава једном.
- Потврда која стигне после одустајања исправља стање (`late_ack_reconciled`).

Задатак уз пример

Додати одлагање између покушаја (`RetryDelay`) које расте са бројем покушаја. Правило додати у `CommandLifecycle` и тестом показати да се поновно слање не дешава пре истека одлагања.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Команде, послови, поновни покушај и идемпотентност". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

ПЗ Тестирање и мануал-цоре-баселине

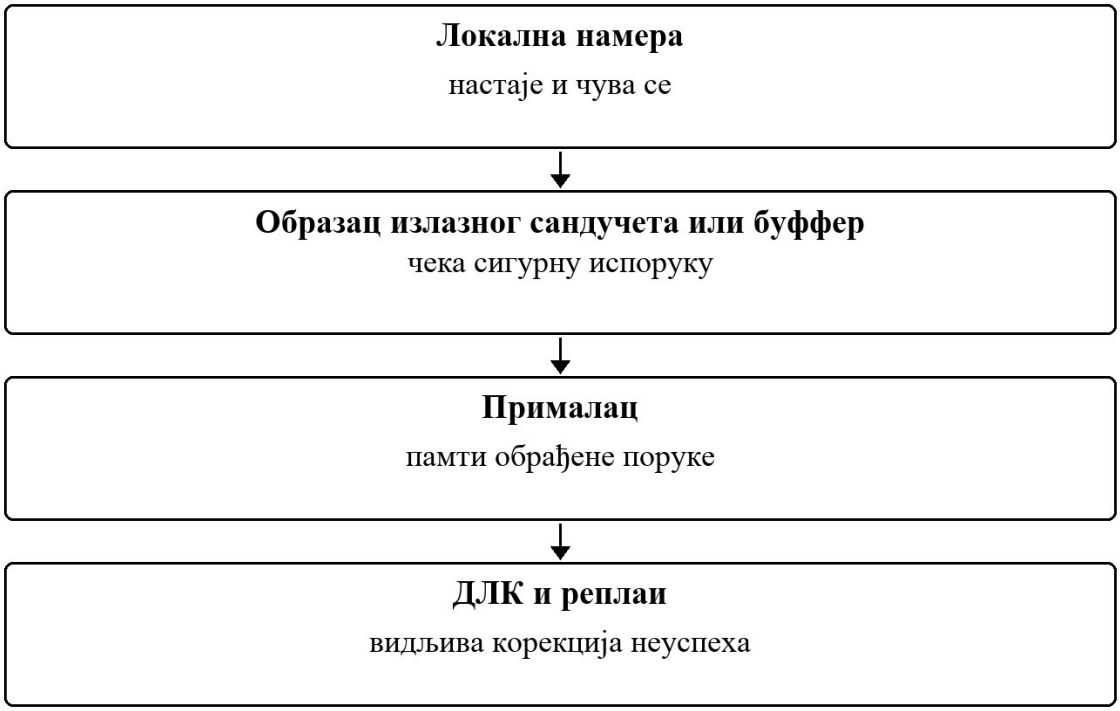
Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.

7 Рад без везе и поуздана испорука

Прекид везе не мора да заустави локални рад, али уводи питање шта се десило док друга страна није била доступна. Буффер, реплаи и образац излазног сандучета обрасци чувају намеру и историју, док дедупликација спречава да поновна испорука створи други пословни ефекат.

Поуздана испорука није обећање да мрежа никада неће погрешити. Она је скуп јасних одговорности који омогућава да се порука сачува, поново пошаље, објасни ако не прође и обради једном са становишта пословног ефекта.



Слика 13. Систем који чува смисао поруке кроз прекид и поновни контакт

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

7.1 Испорука и обрада су одвојене одговорности

- Порука која не пролази не нестаје тихо, већ добија видљив разлог.
- Поновљена испорука је очекивана и прималац је препознаје.
- Локални капацитет и политика одбацивања морају бити експлицитни.

Питање за дискусију

Коју поруку бисте морали сачувати током прекида везе, а коју бисте смели изгубити уз јасно обавештење? Зашто?

7.2 Како се ова тема види у развоју

Тим најпре одлучује шта може да се задржи локално, колико дуго и шта се дешава при попуњеном капацитету. Затим проверава рецоннецт сценарије и показује да се порука може пратити од локалне намере до коначног пословног исхода.

Честе грешке које треба препознати

- Буффер без ограничења и без политике при попуњености.

- Реплаи који заобилази исту валидацију као први покушај.
- Дедупликација ослоњена само на податак који се може променити.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

7.3 Пример: рецоннецт после локалног рада

Удаљена компонента наставља локални рад док нема везу и чува поруке за касније. Када се поново повеже, неке од тих порука су можда већ стигле другим путем. Поуздана испорука зато не значи да се све шаље без провере, већ да се поруке могу идентификовати, поновити и обрађивати без дуплог пословног ефекта.

- Буффер има ограничење и јасно правило када више нема простора.
- Образац излазног сандучета чува намеру заједно са пословном променом коју представља.
- Прималац води евиденцију обрађених порука, а деад-леттер чува неуспехе за контролисан реплаи.

Како се сценарио анализира

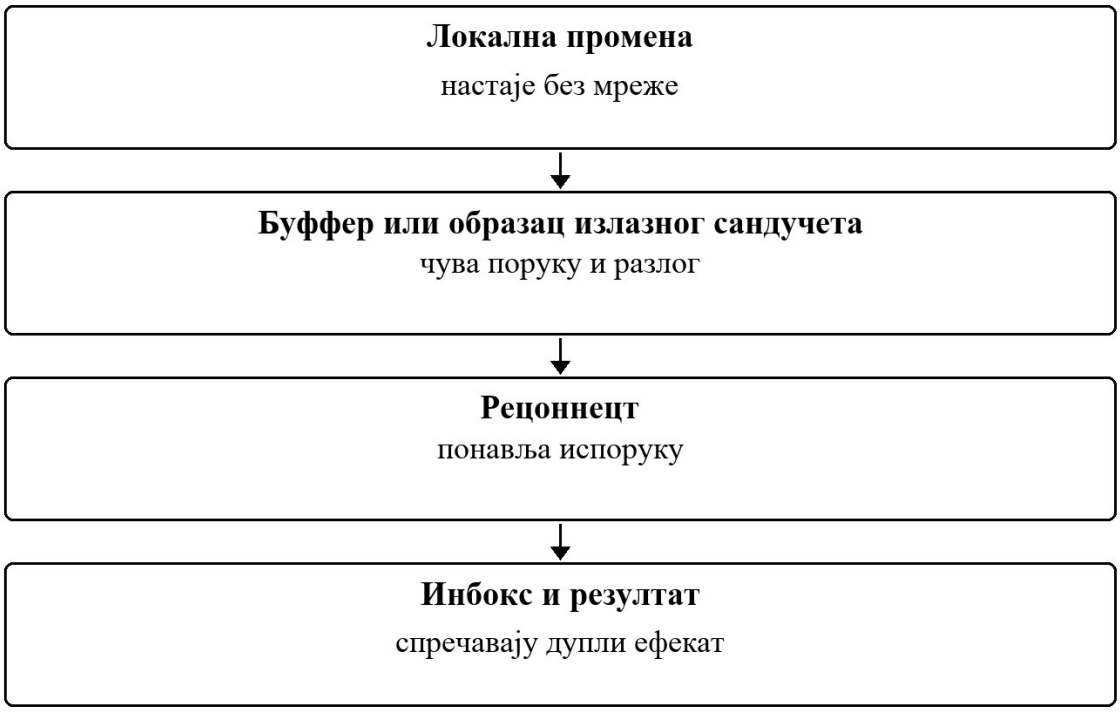
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

7.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

7.5 Шема рада без везе



Слика 14. Шема рада без везе

Прекид везе мења тренутак испоруке, али не сме да избрише намеру нити да је произвољно удвостручи.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преноси и сценарио отказа који може променити ток.

7.6 Упоредни преглед елемената тока

Табела 8. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Локална намера	настаје и чува се	Шта се дешава када локална намера није доступан или даје застарелу информацију?
Образац излазног сандучета или буффер	чека сигурну испоруку	Шта се дешава када образац излазног сандучета или буффер није доступан или даје застарелу информацију?
Прималац	памти обрађене поруке	Шта се дешава када прималац није доступан или даје застарелу информацију?
ДЛК и реплаи	видљива корекција неуспеха	Шта се дешава када длк и реплаи није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

7.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Порука која не пролази не нестаје тихо, већ добија видљив разлог.
2. Поновљена испорука је очекивана и прималац је препознаје.
3. Локални капацитет и политика одбацивања морају бити експлицитни.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Коју поруку бисте морали сачувати током прекида везе, а коју бисте смели изгубити уз јасно обавештење? Зашто?

7.8 Извршиви пример

Пример чува намеру кроз прекид везе. Станица свако мерење најпре уписује у локални образац излазног сандучета, а слање је одвојен корак који се понавља док не стигне потврда. Пошто потврда може да се изгуби и после успешне испоруке, иста порука понекад стигне два пута; центар је зато примењује само први пут.

Пример 25. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-07-rad-bez-veze-outbox
dotnet run --project src/Odp.Vezba07.ConsoleUi
dotnet test Odp.Vezba07.sln
```

Правило у коду

Пример 26. examples/odp-dotnet/vezba-07-rad-bez-veze-outbox/src/Odp.Vezba07.Application/Flushing/FlushOutboxHandler.cs

```
public FlushReport Flush()
{
    var pending = ordering.PendingInOrder(outbox.All());
    if (pending.Count == 0)
        return new(0, 0, OutboxCodes.NothingPending);
```

```
    var delivered = 0;
    foreach (var message in pending)
    {
        if (!uplink.TrySend(message))
            return new(delivered, pending.Count - delivered,
OutboxCodes.DeliveryUnconfirmed);

        outbox.MarkDelivered(message.Position);
        delivered++;
    }

    return new(delivered, 0, OutboxCodes.Flushed);
}
```

Тест који задаје квар

Пример 27. examples/odp-dotnet/vezba-07-rad-bez-veze-outbox/tests/Odp.Vezba07.Tests/Application/OutboxFlowTests.cs

```
[Test]
public void Flush_WhenAckIsLost_ResendsAndTheCenterAppliesTheMessageOnce()
{
    _record.Record("m-1", "a");
    _record.Record("m-2", "b");
    _uplink.LoseNextAck = true;

    var first = _flush.Flush();
    var second = _flush.Flush();

    using (Assert.EnterMultipleScope())
    {
        Assert.That(first, Is.EqualTo(new FlushReport(0, 2,
OutboxCodes.DeliveryUnconfirmed)));
        Assert.That(second, Is.EqualTo(new FlushReport(2, 0, OutboxCodes.Flushed)));
        Assert.That(_center.AppliedPayloads, Is.EqualTo(new[] { "a", "b" }));
        Assert.That(_center.DuplicatesIgnored, Is.EqualTo(1));
    }
}
```

Испис демонстрације

Пример 28. дотнет рун --пројецт src/Odp.Vezba07.ConsoleUi

```
Veza je u prekidu.
  merenje m-1 -> message_queued
  merenje m-2 -> message_queued
  merenje m-3 -> message_queued
  pokušaj slanja bez veze -> delivery_unconfirmed, isporučeno 0, preostalo 3
Veza se vratila, ali se prva potvrda gubi.
  prvi pokušaj posle povratka -> delivery_unconfirmed, isporučeno 0, preostalo 3
  drugi pokušaj -> outbox_flushed, isporučeno 3, preostalo 0

Centar je primenio 3 poruke redom:
  signal -91 dBm
  signal -88 dBm
  signal -97 dBm
Ignorisanih duplikata: 1
```

Шта треба уочити

- Упис мерења успева и када везе нема; слање је одвојен корак.
- После повратка везе поруке стижу редом којим су настале.
- Изгубљена потврда изазива поновно слање, а центар поруку примењује једном.

Задатак уз пример

Ограничити величину образац излазног сандучета-а (`Capacity`). Када је пун, ново мерење се одбија кодом `outbox_full` уместо да се тихо изгуби старо. Правило покрити тестом у коме веза остаје у прекиду.

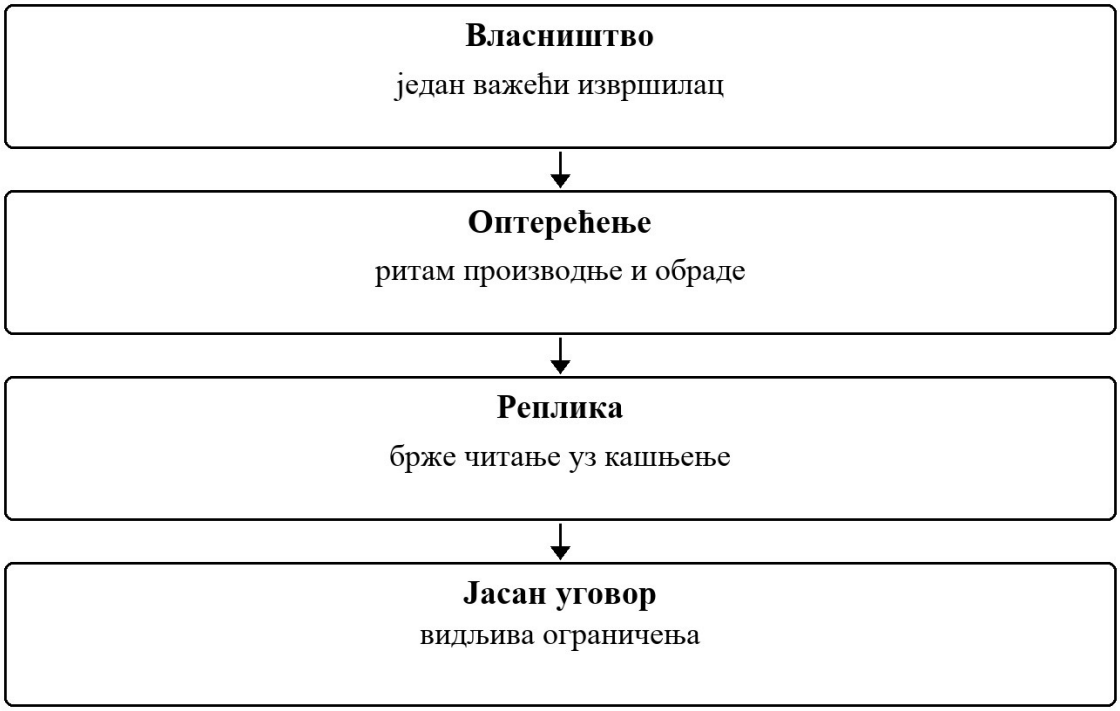
Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Рад без везе и поуздана испорука". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

8 Координација, проток и евентуална конзистентност

Напредни дистрибуирани дизајн прихвата да различите компоненте могу привремено имати различит поглед на стање. Изазов није да се та чињеница сакрије, већ да се власништво, пребацивање рада, оптерећење и свежина података моделују тако да корисник и оператер знају шта систем може да гарантује.

Координација спречава сукоб над ексклузивним радом, повратни притисак штити систем када обрада заостаје, а реплике убрзавају читање уз цену привремене застарелости. Сваки механизам има корист и цену коју тим мора умети да објасни.



Слика 15. Свесни компромиси када систем расте и ради под притиском

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

8.1 Конзистентност је одлука о гаранцији

- Фенцинг токен спречава да стари власник настави рад након преузимања.
- Повратни притисак бира контролисано успоравање или ограничен губитак пре потпуног колапса.
- Модел за читање може бити застарео, али систем мора показати колико и када то утиче на одлуку.

Питање за дискусију

За једну одлуку наведите да ли захтева најновије стање или може да користи застарео приказ, па објасните последицу избора.

8.2 Како се ова тема види у развоју

Завршна одбрана не тражи савршену архитектуру. Тражи јасан компромис: које стање мора бити тренутно, где је прихватљиво кашњење, како се спречава стари власник и шта систем ради када не може да прати улазни ритам.

Честе грешке које треба препознати

- Фаиловер покренут на основу једног кратког прекида.

- Неограничен ред порука који само одлаже квар.
- Коришћење застарелог реад модела као да је ауторитативно стање.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

8.3 Пример: два чвора и један ексклузиван посао

Две инстанце накратко изгубе везу једна са другом. Обе могу мислити да смеју да наставе исти ексклузиван посао. Фенцинг токен или епоха даје новом власнику јачи доказ власништва, па систем може одбити закашнели покушај старе инстанце чак и када се она касније поново појави.

- Координација бира једног важећег извршиоца за рад који не сме бити паралелан.
- Повратни притисак одлучује како систем реагује када улазни ритам надмаши обраду.
- Реплика убрзава читање, али њен приказ мора моћи да покаже застарелост.

Како се сценарио анализира

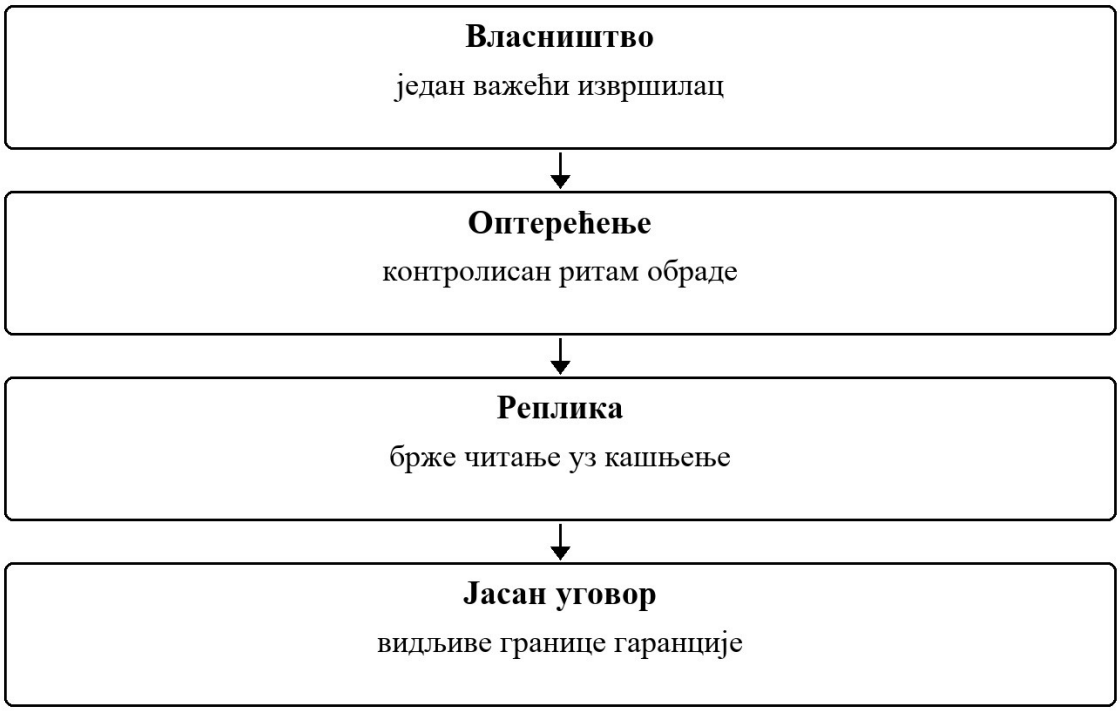
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

8.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

8.5 Шема рада под притиском



Слика 16. Шема рада под притиском

Напредни дистрибуирани дизајн не скрива компромис, већ кориснику и оператеру јасно показује гаранцију.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преносе и сценарио отказа који може променити ток.

8.6 Упоредни преглед елемената тока

Табела 9. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Власништво	један важећи извршилац	Шта се дешава када власништво није доступан или даје застарелу информацију?
Оптерећење	ритам производње и обраде	Шта се дешава када оптерећење није доступан или даје застарелу информацију?
Реплика	брже читање уз кашњење	Шта се дешава када реплика није доступан или даје застарелу информацију?
Јасан уговор	видљива ограничења	Шта се дешава када јасан уговор није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

8.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Фенцинг токен спречава да стари власник настави рад након преузимања.
2. Повратни притисак бира контролисано успоравање или ограничен губитак пре потпуног колапса.
3. Модел за читање може бити застарео, али систем мора показати колико и када то утиче на одлуку.
4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

За једну одлуку наведите да ли захтева најновије стање или може да користи застарео приказ, па објасните последицу избора.

8.8 Извршиви пример

Пример показује две гаранције које систем мора да изабере када расте. Прва је власништво: само један воркер сме да мења распоред станице, власништво је временски ограничено (леасе), а ресурс одбија упис бившег власника по застарелом токену. Друга је проток: када је ред пун, пошиљалац добија јасан одговор уместо да посао неограничено чека.

Пример 29. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-08-koordinacija-protok
dotnet run --project src/Odp.Vezba08.ConsoleUi
dotnet test Odp.Vezba08.sln
```

Правило у коду

Пример 30. examples/odp-dotnet/vezba-08-koordinacija-protok/src/Odp.Vezba08.Domain/Leases/LeasePolicy.cs

```
public LeaseDecision Acquire(Lease? current, string resource, string candidateId,
DateTimeOffset now)
{
    ArgumentException.ThrowIfNullOrWhiteSpace(resource);
    ArgumentException.ThrowIfNullOrWhiteSpace(candidateId);

    if (current is null || now >= current.ExpiresAt)
```

```
    {
        var token = (current?.FencingToken ?? 0) + 1;
        return new(true, LeaseCodes.Granted, new Lease(resource, candidateId, token, now
+ _duration));
    }

    return current.OwnerId == candidateId
        ? new(true, LeaseCodes.Renewed, current with { ExpiresAt = now + _duration })
        : new(false, LeaseCodes.HeldByAnother, current);
}
```

Тест који задаје квар

Пример 31. examples/odp-dotnet/vezba-08-koordinacija-protok/tests/Odp.Vezba08.Tests/Application/CoordinationFlowTests.cs

```
[Test]
public void Write_WhenFormerOwnerWakesUpAfterTakeover_IsRejectedByItsStaleToken()
{
    var former = _acquire.Acquire(Resource, "worker-a").Lease;
    _clock.Advance(TimeSpan.FromSeconds(45));
    var current = _acquire.Acquire(Resource, "worker-b").Lease;
    _write.Write(Resource, current.FencingToken, "plan B");

    var code = _write.Write(Resource, former.FencingToken, "zakasneli plan A");

    using (Assert.EnterMultipleScope())
    {
        Assert.That(code, Is.EqualTo(FencingCodes.StaleToken));
        Assert.That(_schedules.ValueOf(Resource), Is.EqualTo("plan B"));
    }
}
```

Испис демонстрације

Пример 32. дотнет рун --пројецт src/Odp.Vezba08.ConsoleUi

```
Koordinacija:
  worker-a traži lease -> lease_granted (vlasnik worker-a, token 1)
  worker-b traži lease -> lease_held_by_another (vlasnik worker-a, token 1)
  worker-a upisuje "plan A" sa tokenom 1 -> write_accepted
  worker-a je zastao 45 s; lease je istekao
  worker-b traži lease -> lease_granted (vlasnik worker-b, token 2)
  worker-b upisuje "plan B" sa tokenom 2 -> write_accepted
  worker-a upisuje "zakasneli plan A" sa tokenom 1 -> stale_fencing_token
  važeći raspored: plan B

Protok (kapacitet reda 2):
  job-1 -> job_accepted
  job-2 -> job_accepted
  job-3 -> queue_overloaded
```

Шта треба уочити

- У једном тренутку само један воркер је власник ресурса.
- После истека леасе-а нови власник добија већи токен, а упис бившег власника је одбијен.
- Када је ред пун, пошиљалац добија `queue_overloaded` уместо тихог чекања.

Задатак уз пример

Додати у приказ распореда податак о свежини: време последњег уписа и ознаку `IsStale` када је приказ старији од задатог прага. Тестом показати да читалац види застарео, али јасно означен податак док власник не упише нови.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Координација, проток и евентуална конзистентност". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

П4 Оперативни и напредни ниво завршна одбрана

Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.

Сажетак: дистрибуиран систем као објашњив, тестиран процес

Кроз осам вежби гради се један конзистентан ланац: ентитет и власништво → уговор → дистрибуирана операција → failure/recovery сценарио → тест → евиденција. Тај ланац остаје исти без обзира на то да ли се ради о основном хеартбеат механизму, цомманд диспатцх-у или напредном регионалном фаиловер-у — мења се само ниво система на коме се примењује.

Табела 10. Ниво и Шта уводи

Ниво	Шта уводи
P1 — основни	Мисије, станице, ноде-ови, уговори, идентитет, аудит, могућност посматрања система.
P2 — оперативни	Телеметрија, команде, јобс, мессагинг, леасе, ДЛК, рецоннецт.
P3 — напредни	Цоординатион, фаиловер, репликација, повратни притисак, евентуал цонсистенци.

Одговорност остаје код студента

Аутоматизација и ВИ подршка могу убрзати имплементацију, али тим мора разумети дистрибуирани модел, објаснити компромис одабране стратегије и показати независан доказ да failure/recovery понашање заиста ради.

Препоручена литература и документација

Литература служи за продубљивање тема из практикума. Препорука је да се чита уз конкретан дистрибуирани сценарио, јер се принципи најбрже усвајају када студент може да повеже дефиницију са отказ сценаријем, тестом или могућност посматрања система записом.

- Мартин Клеппманн — *Десигнинг Дата-Интенсиве Апплицатионс*.
- Цхрис Рицхардсон — *Мицросервицес Патттернс* (Сага, Образац излазног сандучета, Трансацтионал мессагинг).
- Гоогле СРЕ Боок — *Сите Релиабилитy Енгинееринг*: sre.google/books.
- Мицрософт Леарн — Цлоуд Десигн Паттернс (Поновни покушај, Цирцуит Бреакер, Цомпетинг Цонсумерс): learn.microsoft.com/azure/architecture/patterns.
- Енјунит документација: доцс.Енјунит.орг; Мок документација: github.com/devlooped/moq.
- Пат Хелланд — *Лифе беионд Дистрибутед Трансацтионс* (рад о идемпотенцији и образац излазног сандучета обрасцу).
- Леслие Лампорт — *Тиме, Цлоцкс, анд тхе Ордеринг оф Евентс ин а Дистрибутед Систем* (основа за тему логичког времена и редоследа догађаја).

Напомена о алатима

Конкретни мессаге брокер-и, оркестратори и цлоуд платформе мењају се брже од основних дистрибуираних принципа. Симулатори коришћени у практикуму опонашају њихово понашање ради вежбе — не представљају се као реални производи.